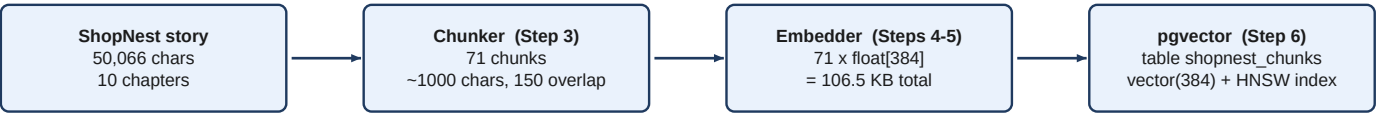


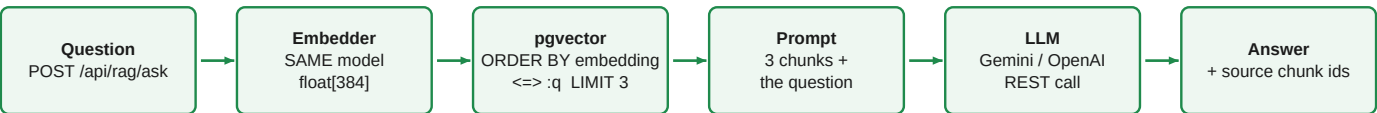
RAG from Scratch, in Java

ShopNest test document → chunks → tokens → 384-number vectors → pgvector → LLM answer. Written for a Spring Boot / JPA / Postgres developer who is new to AI. Lands in **ai-engineering-lab / 03-rag**.

INGEST (run once per document) same model + same vocab for documents AND questions, or the numbers are not comparable



QUERY (every question)



RAG in one paragraph, in Java terms. An LLM is a very smart colleague who has never seen your documents and has a tiny desk (its context window). RAG means: before you ask the colleague, run a `SELECT` that fetches the 3 most relevant paragraphs and paper-clip them to the question. LIKE `'%word%'` cannot find 'relevant' because the question and the document use different words ('mobile cover' vs 'phone case'). So we turn every paragraph into **384 numbers** (an *embedding*) such that paragraphs with similar *meaning* get similar numbers, store them in Postgres, and at question time run `ORDER BY distance LIMIT 3`. The LLM only does the last step: writing the answer from those 3 paragraphs.

AI word	Think of it as (Java / Spring)
Chunk	one page of a paginated result: a fixed-size unit you can process and index
Token	a 'word piece' from the model's tokenizer. The model's limits are counted in tokens, like a buffer is counted in bytes, not characters
Vocabulary	a <code>HashMap<String, Integer></code> with 30,522 entries (line N of vocab.txt = id N)
Embedding	a <code>hashCode()</code> that preserves meaning: a <code>float[384]</code> where nearby arrays mean similar things
Cosine distance (<=>)	a <code>compareTo()</code> for meaning: 0 = same direction, 1 = unrelated, 2 = opposite. Similarity = 1 - distance
Vector index (HNSW)	a B-tree for 'nearest' instead of 'equals'. Approximate, but fast at millions of rows
Context window	the LLM's heap: fixed size, and everything you send (chunks + question + answer) must fit
Prompt template	<code>String.format()</code> / a text block with holes for the context and the question
Retrieval vs generation	retrieval = repository layer (find the rows). Generation = service layer (format the answer from those rows)

Two numbers people mix up (you wrote both: 512 and 256)
The BERT network inside all-MiniLM-L6-v2 has 512 position slots, but sentence-transformers sets **max_seq_length = 256** for this model (its model-card default) and the model was fine-tuned on short text. Anything past 256 word pieces is **silently discarded**. Design for 256, not 512. Output size is always **384** floats, whatever the input length.

What your repo already has vs what 03-rag adds. I read the README of ai-engineering-lab: Milestone 1 (embeddings → pgvector → semantic search) works with Spring AI's `TransformersEmbeddingModel` and `PgVectorStore`, and has no LLM yet. Spring AI hides steps 4-6 behind two beans. This guide builds them by hand once so you can see inside, then you can port the pieces back.

Step	From scratch (this guide)	Already in your lab (Spring AI)
3 split	SentenceChunker (tested here)	nothing. Careful: Spring AI's <code>TokenTextSplitter</code> counts OpenAI-style tokens, not MiniLM word pieces; its default chunk size (800 tokens in 1.x, check 2.0) is over 3x what MiniLM can see
4-5 tokenize + embed	WordPiece + <code>OnnxEmbeddingClient</code> (or HF API / FastAPI)	<code>TransformersEmbeddingModel</code> : same ONNX model, does tokenize + pool for you
6 store + search	ChunkEntity / ChunkRepository + SQL	<code>PgVectorStore</code> (its own table)
7 generate	RagService , PromptBuilder , GeminiClient	<code>ChatClient</code> , not used yet

What is verified and what is not (read this before copy-pasting)

- **Run and tested here:** SentenceChunker, WordPiece tokenizer, pooling, batching, vector literal. 14 self-tests pass on the real 50,066-char doc. My tokenizer produced **identical ids to Hugging Face's tokenizers library on all 71 chunks** (0 mismatches, same vocab file for both). All numbers in this PDF come from those runs. **Vocab caveat:** Hugging Face is blocked here, so I rebuilt the standard bert-base-uncased vocab (30,522 entries) from an npm package. all-MiniLM-L6-v2 uses that vocabulary as far as I know, but diff it against the vocab.txt in the model repo; if it differs, my token counts shift slightly.
- **Written against the APIs but NOT compiled** (this sandbox cannot reach Maven Central): ONNX, RestClient, JPA entity, Gemini and controller code. Expect to fix an import or a method name. Each such block says what to double-check.
- **Not run:** the real embedding model (Hugging Face is blocked here). So there are no real similarity scores in this PDF. The 'expected chunks' table is from exact string search; your own run tells you what the model actually retrieves.

STEP 1 STORY DOC CREATION: the ShopNest test document

The file **shopnest_story.txt** (delivered next to this PDF) is a continuous technical story: **50,066 characters, 98 paragraphs, 387 sentences, 10 chapters**. Every chapter ends with a sentence that leads into the next one. It is fiction, written to be a good RAG test bed: exact numbers to check answers against, one fact stated twice (the first order appears in Chapters 1 and 10), abbreviations and decimals that break naive sentence splitters ('Dr. Meera Rao', 'Rs. 9.6 lakh'), and questions the text cannot answer.

#	Chapter	Chars	A fact you can test
1	The Garage Idea	4,944	first order: Guntur chillies, Rs. 310
2	The First Monolith Shows Its Cracks	5,126	N+1 fix: 151 statements down to 3
3	Payments, Trust and the Double Charge	5,177	double charge: 37 customers, no idempotency
4	When Search Returns Nothing	5,081	search: 31% of queries returned zero results
5	The Diwali Surge	4,841	Diwali outage: 47 minutes
6	Breaking the Monolith with Kafka	5,035	orders topic: 12 partitions
7	Observability and the First On-Call Rota	4,845	checkout SLO: 99.9 percent
8	The Cloud Bill Shock	4,973	AWS bill: Rs. 9.6 lakh to Rs. 5.8 lakh
9	Teaching Search to Understand	5,231	all-MiniLM-L6-v2, 384 dims, 256 word pieces
10	Growing the Team and Looking Ahead	4,794	Series A: Rs. 42 crore

Answer key (chunk numbers are for the Step 3 chunker with 1000/150; if you re-implement the chunker, match on *content*, not on ids):

Question you ask	Retrieved text must contain	Expected chunk(s)
What was the first order and how much was it?	Guntur chillies	4 and 68 (chapters 1 and 10)
How many customers were charged twice, and why?	Thirty seven customers	15
How was the N+1 query problem fixed?	JOIN FETCH (and 151 to 3)	8 and 9
How long did the Diwali outage last?	The outage lasted 47 minutes	29 and 30 (sentence sits in the overlap)
How many partitions does the orders topic have?	12 partitions	37
What availability target was set for checkout?	99.9 percent	45
How much did the AWS bill fall by?	Rs. 9.6 lakh to Rs. 5.8 lakh	53
Which embedding model, how many dimensions?	all-MiniLM-L6-v2	55 and 56
How much did ShopNest raise in its Series A?	Rs. 42 crore	63 and 64
What was the real return policy?	seven days	59
Who is ShopNest's CFO? (not in the document)	nothing	none: the app must say 'I don't know'

Why this matters

You cannot judge RAG on a document you have not memorised. A test document with known answers is your unit-test dataset: 'is the right chunk in the top 3?' becomes a yes/no check you can automate (Step 8 has the harness).

The unanswerable question is the most important one. A RAG system that never says 'I don't know' is a hallucination machine.

What goes wrong if you skip this

- You test with 2-3 casual questions, they happen to work, and you ship. Later a chunk-size change quietly breaks 3 of 10 questions and nobody notices.
- You test only questions that have answers, so the 'garbage in, confident answer out' failure never shows up.

STEP 2 WHY WE SPLIT: you cannot embed 50k characters at once

Reason 1: a hard input limit. The model is a function whose input is a fixed-size array of token ids. For all-MiniLM-L6-v2 the useful limit is 256. In Java terms this is not the heap (`OutOfMemoryError`) or the stack (`StackOverflowError`): those fail loudly. It behaves like `Arrays.copyOf(buffer, 256)`: the rest is **silently chopped off, no exception, no log line**. That is the worst kind of limit.

Reason 2: cost grows with the square. Self-attention compares every token with every other token: an $n \times n$ table, a nested for-loop. 256 tokens = 65,536 cells. 512 tokens = 262,144 cells (4x). Our whole document, if the model accepted it, would be 10,185 tokens = 103,734,225 cells, about **1,583x** the work of one 256-token input, for one single vector.

Reason 3: quality. Even with no limit, one vector for the whole document is the *average meaning of everything*. It is like calling `hashCode()` on an entire library and trying to find the right shelf. Small chunks give sharp vectors, precise retrieval, and a small prompt (3 chunks of ~1000 chars is ~600 tokens instead of 10,000).

Calculation	Working	Result
Rule of thumb (1 token = 4 chars)	50,066 / 4	~12,500 tokens (rough)
Measured with the real WordPiece vocab	my tokenizer, whole document, incl. [CLS]/[SEP]	10,185 tokens (4.92 chars per token)
Minimum chunks if packed perfectly	10,185 / 256 = 39.8	at least 40
Actual chunks (1000 chars, 150 overlap, whole sentences)	avg chunk = 920 chars = 190 tokens	71 chunks
Why 71 and not 40	1000 chars is only ~190 tokens (headroom under 256) + repeated overlap text	65,300 chars stored vs 50,066 (+30%)
Vector storage	71 x 384 floats x 4 bytes	109,056 bytes = 106.5 KB
If you embedded the doc as ONE input	256 / 10,185	model sees 2.5%; 97.5% never indexed

Share of each chunk's tokens the model actually embeds (limit: 256 tokens per input)

5000-char windows 10 chunks, avg 1,021 tokens	25%	75% of the text is invisible
3000-char windows 17 chunks, avg 602 tokens	43%	57% invisible
1000 chars, sentence-aware (ours) 71 chunks, avg 190, max 223 tokens	100%	nothing dropped
500-char windows 100 chunks, avg 105 tokens	100%	nothing dropped, but fragments
200-char windows 250 chunks, avg 44 tokens	100%	nothing dropped, but 4 words of context

blue = embedded grey = silently discarded by truncation

Why this matters

Chunk size is a design constraint set by the *model*, not a style choice: chunk tokens must stay under 256 or the tail is lost. 1000 English chars is about 190-225 tokens here, which fits. Our worst chunk is 223 tokens, so there is a 33-token safety margin. Technical text with identifiers (HikariCP = 3 tokens, Kondapur = 3 tokens) packs more tokens per character than plain prose, so always measure.

What goes wrong if you do not split (or split too big)

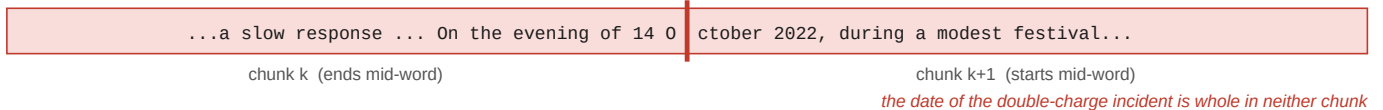
- **No split:** the model embeds the first ~1,250 characters (inside Chapter 1) and nothing else. 'How much was the Series A?' (Chapter 10) can never be found.
- **5000-char chunks:** measured on this document: 10 chunks, every one over 256 tokens, **only 25% of the text is embedded**. Retrieval looks like it works (results come back) but is blind to three quarters of every chunk.
- **Too small (200 chars):** 250 chunks of ~44 tokens: no context, 'it' and 'the team' lose their meaning, top-3 returns three fragments.

STEP 3 SPLITTING LOGIC IN JAVA: 1000 chars, 150 overlap, sentence-aware

Not 'just chunk it'. The algorithm, using only String / List methods:

- **1. Split into paragraphs** on blank lines (headings like 'Chapter 2: ...' become their own sentence).
- **2. Split each paragraph into sentences** by scanning for . ! ? and asking `isBoundary()`: the next char must be a space, the word before must not be an abbreviation (Dr, Rs, Mr, e.g...) or a single initial, and the next sentence must start with an uppercase letter, digit or quote. So '3.2', 'Rs. 18 lakh' and 'Dr. Meera Rao' survive.
- **3. Pack sentences greedily** into a window until adding the next one would exceed 1000 chars; emit the chunk.
- **4. Carry the tail forward**: walk backwards over whole sentences until at least 150 chars are collected; these start the next chunk (the overlap).
- **5. Edge cases**: a single sentence longer than 1000 chars is cut at a space (`lastIndexOf(' ', max)`); if overlap + next sentence would not fit, the overlap is dropped for that chunk.

A) Fixed 1000-char windows, no overlap (what `substring(i, i+1000)` gives you) - real casualty from our doc:



B) Sentence-aware + whole-sentence overlap (ours) - real chunks 29 and 30:



Requests began to time out after 30 seconds, the load balancer marked instances unhealthy, and at 8:23 the site went down completely. The outage lasted 47 minutes. During the incident the team discovered three compounding problems.

SentenceChunker.java (tested: this exact file produced every number in this guide)

```
import java.util.ArrayList;
import java.util.List;
import java.util.Set;

/** Sentence-aware chunker: ~maxChars per chunk, whole-sentence overlap of at least overlapChars. */
public class SentenceChunker {

    private static final Set<String> ABBREV =
        Set.of("dr", "mr", "mrs", "ms", "rs", "vs", "e.g", "i.e", "etc", "approx", "inc", "ltd");

    private final int maxChars;
    private final int overlapChars;

    public SentenceChunker(int maxChars, int overlapChars) {
        if (overlapChars >= maxChars) throw new IllegalArgumentException("overlap must be < chunk size");
        this.maxChars = maxChars;
        this.overlapChars = overlapChars;
    }

    public List<String> chunk(String text) {
        List<String> sentences = splitSentences(text);
        List<String> chunks = new ArrayList<>();
        List<String> window = new ArrayList<>();
        int len = 0;
        // sentences in the chunk being built
        // joined length of window (single spaces)

        for (String s : sentences) {
            int add = (window.isEmpty() ? 0 : 1) + s.length();
            if (len + add > maxChars && !window.isEmpty()) {
                chunks.add(String.join(" ", window));
                window = overlapTail(window);
                len = joinedLength(window);
                // 1. emit the full chunk
                // 2. carry the last sentences forward
            }
            if (len + 1 + s.length() > maxChars) {
                window.clear();
                len = 0;
                // 3. overlap + new sentence too big? drop overlap
            }
            window.add(s);
            len = joinedLength(window);
        }
        if (!window.isEmpty()) chunks.add(String.join(" ", window));
        return chunks;
    }
}
```

```

/** Walk backwards, collecting WHOLE sentences until we have >= overlapChars. */
private List<String> overlapTail(List<String> window) {
    List<String> tail = new ArrayList<>();
    int n = 0;
    for (int i = window.size() - 1; i >= 0 && n < overlapChars; i--) {
        tail.add(0, window.get(i));
        n += window.get(i).length() + (tail.size() > 1 ? 1 : 0);
    }
    return n > maxChars / 2 ? new ArrayList<>() : tail;    // a monster sentence: skip overlap
}

static int joinedLength(List<String> sentences) {
    int n = 0;
    for (String s : sentences) n += s.length();
    return n + Math.max(0, sentences.size() - 1);
}

// ----- sentence splitting with plain String methods -----

List<String> splitSentences(String text) {
    List<String> out = new ArrayList<>();
    for (String para : text.split("\n\n\s*\n")) {                // paragraphs first
        para = para.trim().replaceAll("\s+", " ");
        int start = 0;
        for (int i = 0; i < para.length(); i++) {
            char c = para.charAt(i);
            if ((c == '.' || c == '!' || c == '?') && isBoundary(para, i)) {
                addSentence(out, para.substring(start, i + 1).trim());
                start = i + 1;
            }
        }
        if (start < para.length()) addSentence(out, para.substring(start).trim());
    }
    return out;
}

private static boolean isBoundary(String p, int i) {
    if (i + 1 >= p.length()) return true;                        // end of paragraph
    if (p.charAt(i + 1) != ' ') return false;                    // "3.2", "v2.1", "e.g.cat"
    String word = p.substring(p.lastIndexOf(' ', i) + 1, i).toLowerCase();
    if (ABBREV.contains(word)) return false;                     // "Dr. Rao", "Rs. 18 lakh"
    if (word.length() == 1 && Character.isLetter(word.charAt(0))) return false; // "K. Varma"
    if (i + 2 >= p.length()) return true;
    char next = p.charAt(i + 2);
    return Character.isUpperCase(next) || Character.isDigit(next) || next == '"';
}

private void addSentence(List<String> out, String s) {
    if (s.isEmpty()) return;
    while (s.length() > maxChars) {                             // monster sentence: split on a space
        int cut = s.lastIndexOf(' ', maxChars);
        if (cut <= 0) cut = maxChars;
        out.add(s.substring(0, cut));
        s = s.substring(cut).trim();
    }
    if (!s.isEmpty()) out.add(s);
}
}

```

Strategy on the 387-sentence document	Chunks	Sentences cut in half and never whole anywhere	Chunks starting mid-sentence
Fixed 1000 chars, no overlap (substring)	50	46 of 387 (12%)	48
Fixed 1000 chars + 150-char overlap	59	4 of 387	58
Sentence-aware, 1000 chars, >= 150 overlap (ours)	71	0	0

Overlap as measured: because we carry whole sentences, the repeated text between neighbours is at least 150 chars; across the 70 chunk boundaries it averages 219 chars (min 150, max 412). A character-exact 150 would cut sentences, so 'at least 150, rounded up to whole sentences' is the sentence-safe reading of your spec. **Splitter trap:** a naive `split("(?<=\\.)\s")` finds 392 'sentences' instead of 387 and produces junk fragments such as 'Dr.', '310.' and '6,400.'. **Known gaps** of our splitter: a short abbreviation list, no handling of 'U.S. Army', and text without periods (tables, code, bullet lists) becomes one huge 'sentence' that falls back to the hard cut. For real documents, also split on Markdown headings first.

JUnit 5 tests to keep next to the chunker (not compiled here; the same invariants ran in my plain-Java SelfTest)

```
// SentenceChunkerTest.java (plain JUnit 5; these exact invariants all passed in my run of the real doc)
class SentenceChunkerTest {
    final String text = read("shopnest_story.txt");
    final SentenceChunker chunker = new SentenceChunker(1000, 150);
    final List<String> chunks = chunker.chunk(text);

    @Test void noChunkExceedsLimit() { assertTrue(chunks.stream().allMatch(c -> c.length() <= 1000)); }
    @Test void noBlankChunks() { assertTrue(chunks.stream().noneMatch(String::isBlank)); }
    @Test void everySentenceStaysWhole() {
        assertTrue(chunker.splitSentences(text).stream().allMatch(s -> chunks.stream().anyMatch(c -> c.contains(s))));
    }
    @Test void consecutiveChunksOverlap() {
        for (int i = 0; i + 1 < chunks.size(); i++) assertTrue(overlap(chunks.get(i), chunks.get(i + 1)) >= 150);
    }
    @Test void abbreviationsAreNotBoundaries() {
        assertTrue(chunker.splitSentences("Dr. Rao paid Rs. 18 lakh. Then she left.").size() == 2);
    }
    @Test void tokenLimitRespected(WordPiece wp) {
        assertTrue(chunks.stream().allMatch(c -> wp.countTokens(c) <= 256));
    }
}
```

Why overlap keeps relevance

An embedding describes what is *inside* a chunk. A fact that straddles a boundary is half in each chunk, so **neither** chunk 'means' the fact and both score low for the question. Repeating the boundary sentences guarantees that at least one chunk contains the whole fact. In our doc, 'The outage lasted 47 minutes.' sits at the boundary of chunks 29 and 30 and is complete in both. Cost: about 30% more chunks to embed and store, and neighbouring chunks may both appear in the top 3 (accept it or de-duplicate).

What goes wrong without it

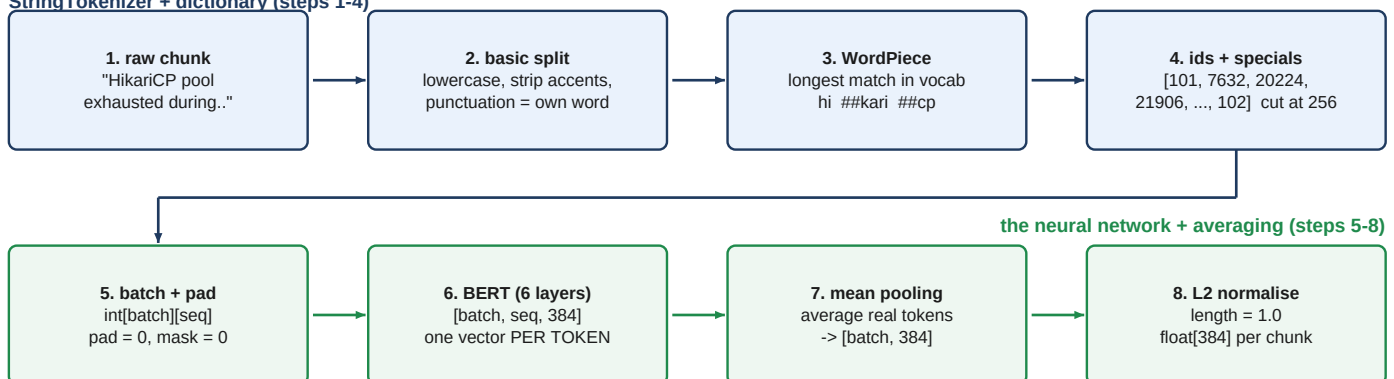
- **No overlap, fixed windows:** 46 of 387 sentences (12%) exist whole nowhere. Real casualties from the doc: 'On the evening of 14 October 2022' (the double-charge date) and 'touched the product of each item' (the N+1 explanation).
- **Cut mid-sentence:** a chunk starting with 'ctober 2022, during a modest festival...' gives the model (and the LLM later) garbage context.
- **Spring AI TokenTextSplitter with its defaults** (800-token chunks in 1.x as far as I know; check 2.0, and it counts OpenAI-style tokens): roughly 3x the 256 limit, so most of every chunk would be truncated. Set the size explicitly and verify with a token count.

STEP 4 TOKENIZATION DEEP DIVE: all-MiniLM-L6-v2 and WordPiece

A neural network cannot read characters; it reads integers. The **tokenizer** turns text into integers, and it is closer to a `StringTokenizer` plus a dictionary lookup than to anything mysterious. WordPiece works like this: lowercase the text, strip accents, split on spaces and punctuation, then for each word find the **longest prefix that exists in the vocabulary**, emit it, and repeat on the remainder, marking continuation pieces with `##`. A word the vocabulary has never seen does not fail: it is broken into smaller known pieces (HikariCP → hi ##kari ##cp). Only if no piece matches does it become `[UNK]`.

The vocabulary is a plain text file, **vocab.txt**, 30,522 lines, where **line number = token id**: `[PAD]=0`, `[UNK]=100`, `[CLS]=101`, `[SEP]=102`, `the=1996`, `shop=4497`. `[CLS]` and `[SEP]` are added at the start and end of every input. Take `vocab.txt` from the model's Hugging Face repo for your build (my numbers used the `bert-base-uncased` vocab; diff the two files once).

StringTokenizer + dictionary (steps 1-4)



WordPiece.java (tested: identical ids to Hugging Face tokenizers on all 71 chunks and on the whole doc)

```
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.text.Normalizer;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/** Minimal BERT (uncased) WordPiece tokenizer: the same algorithm all-MiniLM-L6-v2 uses. */
public class WordPiece {

    public static final int PAD = 0, UNK = 100, CLS = 101, SEP = 102;
    private final Map<String, Integer> vocab = new HashMap<>();
    private final List<String> inverse = new ArrayList<>();

    public WordPiece(Path vocabTxt) throws IOException {
        for (String line : Files.readAllLines(vocabTxt)) { // line number == token id
            vocab.put(line, inverse.size());
            inverse.add(line);
        }
    }

    /** text -> word pieces, e.g. "HikariCP" -> [hi, ##kari, ##cp] */
    public List<String> tokenize(String text) {
        List<String> pieces = new ArrayList<>();
        for (String word : basicSplit(text)) {
            pieces.addAll(wordPiece(word));
        }
        return pieces;
    }

    /** Ids with [CLS] ... [SEP], truncated to maxlen (keeps the final [SEP]). */
    public int[] encode(String text, int maxlen) {
        List<String> pieces = tokenize(text);
        int n = Math.min(pieces.size(), maxlen - 2);
        int[] ids = new int[n + 2];
        ids[0] = CLS;
        for (int i = 0; i < n; i++) ids[i + 1] = vocab.getOrDefault(pieces.get(i), UNK);
        ids[n + 1] = SEP;
        return ids;
    }

    public int countTokens(String text) { // no truncation, includes [CLS]/[SEP]
        return tokenize(text).size() + 2;
    }

    public String idToToken(int id) { return inverse.get(id); }

    // 1) lowercase, strip accents, split on whitespace AND punctuation (each punctuation char is its own word)
    private List<String> basicSplit(String text) {
        String t = Normalizer.normalize(text.toLowerCase(), Normalizer.Form.NFD).replaceAll("\\p{Mn}", "");
        List<String> words = new ArrayList<>();
        StringBuilder cur = new StringBuilder();
        for (char c : t.toCharArray()) {
            if (Character.isWhitespace(c)) { flush(words, cur); }
            else if (isPunct(c)) { flush(words, cur); words.add(String.valueOf(c)); }
            else { cur.append(c); }
        }
        flush(words, cur);
        return words;
    }

    private static void flush(List<String> words, StringBuilder cur) {
        if (cur.length() > 0) { words.add(cur.toString()); cur.setLength(0); }
    }

    private static boolean isPunct(char c) {
        return (c >= 33 && c <= 47) || (c >= 58 && c <= 64) || (c >= 91 && c <= 96) || (c >= 123 && c <= 126)
            || Character.getType(c) == Character.OTHER_PUNCTUATION
            || Character.getType(c) == Character.DASH_PUNCTUATION;
    }

    // 2) greedy longest-match-first against the vocabulary; continuation pieces get a "##" prefix
    private List<String> wordPiece(String word) {
        if (word.length() > 100) return List.of("[UNK]");
        List<String> out = new ArrayList<>();
        int start = 0;
        while (start < word.length()) {
            int end = word.length();
            String found = null;
            while (start < end) {
                String sub = word.substring(start, end);
                if (start > 0) sub = "##" + sub;
                if (vocab.containsKey(sub)) { found = sub; break; }
                end--;
            }
            if (found == null) return List.of("[UNK]"); // whole word becomes [UNK]
            out.add(found);
            start = end;
        }
        return out;
    }
}
```


Debugging one real chunk (chunk 29, 940 chars). This is the actual output of my Debug.java, the kind of thing you would see stepping through in IntelliJ:

Console output

```
== A. one chunk ==
chars=940 wordpieces=197 +[CLS]/[SEP] = 199 (##continuations=18, [UNK]=0)
chars per token = 4.72
first 16: [CLS](101) ka(10556) ##rth(15265) ##ik(5480) ,(1010) remembering(10397) the(1996) festival(2782)
         promotion(4712) of(1997) the(1996) previous(3025) year(2095) ,(1010) insisted(7278) on(2006)
last 4:  ##ing(2075) problems(3471) .(1012) [SEP](102)

== B. how words are cut ==
HikariCP      -> [hi, ##kari, ##cp]
Kondapur      -> [ko, ##nda, ##pur]
idempotency   -> [id, ##em, ##pot, ##ency]
pgvector      -> [pg, ##ve, ##ctor]
Diwali        -> [di, ##wal, ##i]
N+1           -> [n, +, 1]
Rs. 9.6 lakh  -> [rs, ., 9, ., 6, la, ##kh]
ShopNest      -> [shop, ##nes, ##t]

== C. padding a batch of 3 ==
"Diwali sale outage"          ids=[101, 4487, 13476, 2072, 5096, 2041, 4270, 102, 0, 0]
                               mask=[1, 1, 1, 1, 1, 1, 1, 1, 0, 0]
"HikariCP pool exhausted during the sale" ids=[101, 7632, 20224, 21906, 4770, 9069, 2076, 1996, 5096, 102]
                               mask=[1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
"Orders"                     ids=[101, 4449, 102, 0, 0, 0, 0, 0, 0, 0]
                               mask=[1, 1, 1, 0, 0, 0, 0, 0, 0, 0]

== D. silent truncation at 256 ==
text chars=2723 real tokens=542 model sees=256 silently dropped=286 tokens (52%)
last token the model sees: 'un' first dropped tokens: ##ca ##ched query for trend ##ing products on ...
== E. mean pooling (toy: hidden size 4 instead of 384) ==
mean over 4 real tokens = [0.54999995, 0.125, 0.050000004, 0.17500001]
L2 length = 0.5927 -> normalized = [0.928014, 0.21091229, 0.08436492, 0.2952772]
(bug: averaging the pad row too = [2.4199998, 2.08, 2.02, 2.12])
```

Section	What it shows	Java way to think about it
A	940 chars -> 197 word pieces + [CLS] + [SEP] = 199 tokens (4.72 chars/token); 18 pieces are '##' continuations; 0 [UNK]. Names like Karthik become ka ##rth ##ik.	token count is the 'length' that matters, not String.length()
B	Rare words are cut into known fragments: HikariCP, Kondapur, idempotency, pgvector, Diwali. 'Rs. 9.6 lakh' becomes 7 tokens.	like decomposing an unknown word into a char[] of known syllables
C	Padding: tensors must be rectangular, so the batch is an int [3] [10]; short rows are filled with 0 ([PAD]) and a parallel attention mask marks real tokens (1) vs padding (0).	a 2-D array cannot be jagged; the mask is a validity bitmap
D	Over the limit: 2,723 chars = 542 tokens; the model sees 256 and 286 (52%) are dropped , cutting inside the word 'uncached' ('un' kept, '##ca ##ched' lost). No warning.	silent substring(0, 256)
E	Mean pooling. The network outputs one 384-vector per token ([seq, 384]). We average the rows of real tokens (mask = 1) into one vector, then scale it to length 1.	stream().average() over rows, skipping nulls

Where does 384 come from? It is the model's hidden size: the width of the vector each layer keeps for every token (6 layers, hence 'L6'; about 22 million parameters, ~90 MB file). It is fixed by the architecture, so a different model means a different dimension (and a different table column). Each chunk costs a float [384] = 1,536 bytes. **Normalising** to length 1 makes cosine similarity equal to a plain dot product; pgvector's <=> ignores length anyway, but normalising lets you switch operators later and keeps scores comparable.

Why this matters

Tokens, not characters, decide what the model sees. Counting tokens (WordPiece.countTokens) lets you log a warning at ingestion when a chunk exceeds 256, and pooling with the mask is what turns a [seq, 384] tensor into the single vector you store.

What goes wrong

- **Ignoring truncation:** chunks over 256 tokens lose their tail silently; retrieval quality drops with no error anywhere (example D: 52% lost).
- **Averaging the padding rows too** (toy example E): the pooled vector came out as [2.42, 2.08, 2.02, 2.12] instead of [0.55, 0.13, 0.05, 0.18], dominated by padding. Same text embedded in a batch of 1 vs a batch of 16 would give different vectors, a nasty bug to find.
- **Different vocab for documents and questions** (or an uncased vs cased mix-up): the ids mean different words, similarities are noise.
- **A tokenizer.json that truncates at 128:** some tokenizer files carry their own max_length. Test that a 200-token input yields ~200 ids, not 128 (I could not download the file here to check).

STEP 5 EMBEDDING IN A JAVA ENVIRONMENT: four ways, one interface

What 'embedding' means operationally: run the neural network forward once (no training): token ids in, float[384] out. Everything in Steps 3-4 is plain Java; only this one step needs the model. Hide it behind an interface (the Strategy pattern, so RagService never changes) and pick an implementation with rag.embedder=...:

	A. ONNX in-process	B. HF Inference API	C. Python FastAPI	D. Spring AI bean
Where the model runs	inside your JVM	Hugging Face servers	a local Python process	inside your JVM (what your lab uses)
Extra setup	onnxruntime jar + model.onnx + vocab.txt	HF token, internet	pip install + run uvicorn	none, already wired
Tokenizer	our WordPiece (Step 4)	server side	server side (Python)	inside Spring AI (DJL)
Offline / private	yes	no : text leaves your machine	yes	yes
8 GB laptop	fine: ~90 MB model, plus the native runtime	nothing local	Python + torch: heavier (GBs of install)	fine; the 'heavy native runtime' you noticed
Pick it when	you want to understand ONNX and keep one process	quick experiment, no local install	you want the reference implementation to compare against	simplest for the lab; use it as the oracle

Shared pieces: interface, batching, pooling, vector literal (all four files compiled and tested here)

```
// ===== EmbeddingClient.java =====
import java.util.List;

/** One interface, several implementations (ONNX in-process, HF API, FastAPI, Spring AI wrapper). */
public interface EmbeddingClient {

    int DIMENSIONS = 384;

    /** Embeds all texts. Result order MUST match input order. Vectors are L2-normalised. */
    List<float[]> embedAll(List<String> texts);

    default float[] embed(String text) {
        return embedAll(List.of(text)).get(0);
    }
}

// ===== Batches.java =====
import java.util.ArrayList;
import java.util.List;

public final class Batches {
    private Batches() {}

    /** 50 items, size 16 -> [16, 16, 16, 2]. Views, not copies. */
    public static <T> List<List<T>> partition(List<T> items, int size) {
        List<List<T>> out = new ArrayList<>();
        for (int i = 0; i < items.size(); i += size) {
            out.add(items.subList(i, Math.min(i + size, items.size())));
        }
        return out;
    }
}

// ===== Pooling.java =====
public final class Pooling {
    private Pooling() {}

    /** hidden: [seq][384] from the transformer, mask: 1 = real token, 0 = [PAD]. Returns one unit-length 384-vector. */
    public static float[] meanAndNormalize(float[][] hidden, long[] mask) {
        int dim = hidden[0].length;
        float[] sum = new float[dim];
        int count = 0;
        for (int t = 0; t < hidden.length; t++) {
            if (mask[t] == 0) continue; // padding must NOT contribute
            for (int d = 0; d < dim; d++) sum[d] += hidden[t][d];
            count++;
        }
        for (int d = 0; d < dim; d++) sum[d] /= Math.max(count, 1);
        return normalize(sum);
    }

    /** Divide by the L2 length so cosine similarity == dot product. */
    public static float[] normalize(float[] v) {
        double norm = 0;
        for (float x : v) norm += x * x;
        norm = Math.max(Math.sqrt(norm), 1e-12);
        float[] out = new float[v.length];
        for (int i = 0; i < v.length; i++) out[i] = (float) (v[i] / norm);
    }
}
```

```

        return out;
    }
}
// ===== VectorLiteral.java =====
public final class VectorLiteral {
    private VectorLiteral() {}

    /** float[] -> "[0.1,0.2,...]" : the text format pgvector accepts. */
    public static String of(float[] v) {
        StringBuilder sb = new StringBuilder(v.length * 10).append('[');
        for (int i = 0; i < v.length; i++) {
            if (Float.isNaN(v[i]) || Float.isInfinite(v[i])) throw new IllegalArgumentException("bad value at " + i);
            if (i > 0) sb.append(',');
            sb.append(v[i]);
        }
        return sb.append(']').toString();
    }
}
}

```

Option A: ONNX Runtime from Java. Files you need: `model.onnx` (the Xenova/all-MiniLM-L6-v2 export has `onnx/model.onnx`; the original sentence-transformers repo also ships an `onnx/` folder, check the file list) and `vocab.txt`. The model's output is `last_hidden_state` [batch, seq, 384]; pooling and normalising are **your** job, which is why Pooling exists. Print `session.getInputNames()` once to confirm the three input names.

OnnxEmbeddingClient.java (written against the onnxruntime Java API, not compiled here)

```

// OnnxEmbeddingClient.java -- Option A: the model runs INSIDE your JVM (no network, no Python)
// pom: com.microsoft.onnxruntime:onnxruntime (plain CPU build)
import ai.onnxruntime.*;
import java.nio.file.Path;
import java.util.*;

@Service
@ConditionalOnProperty(name = "rag.embedder", havingValue = "onnx")
public class OnnxEmbeddingClient implements EmbeddingClient, AutoCloseable {

    private static final int MAX_LEN = 256; // sentence-transformers default for all-MiniLM-L6-v2
    private static final int BATCH = 16;

    private final OrtEnvironment env = OrtEnvironment.getEnvironment();
    private final OrtSession session;
    private final WordPiece tokenizer; // our class from Step 4 (same ids as HF tokenizers)

    public OnnxEmbeddingClient(@Value("${rag.onnx.model}") String modelPath, WordPiece tokenizer) throws OrtException {
        this.session = env.createSession(modelPath, new OrtSession.SessionOptions());
        this.tokenizer = tokenizer;
        // DEBUG ONCE: System.out.println(session.getInputNames()); expect [input_ids, attention_mask, token_type_ids]
    }

    @Override
    public List<float[]> embedAll(List<String> texts) {
        List<float[]> out = new ArrayList<>(texts.size());
        for (List<String> batch : Batches.partition(texts, BATCH)) out.addAll(embedBatch(batch));
        return out;
    }

    private List<float[]> embedBatch(List<String> batch) {
        int[][] enc = new int[batch.size()][];
        int seq = 0;
        for (int i = 0; i < enc.length; i++) {
            enc[i] = tokenizer.encode(batch.get(i), MAX_LEN); // [CLS] ... [SEP], truncated
            seq = Math.max(seq, enc[i].length);
        }
        long[][] ids = new long[enc.length][seq]; // zero-filled == [PAD]
        long[][] mask = new long[enc.length][seq]; // 1 = real token
        long[][] types = new long[enc.length][seq]; // all 0: single-sentence input
        for (int b = 0; b < enc.length; b++) {
            for (int t = 0; t < enc[b].length; t++) { ids[b][t] = enc[b][t]; mask[b][t] = 1; }
        }
        try (OnnxTensor tIds = OnnxTensor.createTensor(env, ids);
            OnnxTensor tMask = OnnxTensor.createTensor(env, mask);
            OnnxTensor tTypes = OnnxTensor.createTensor(env, types);
            OrtSession.Result res = session.run(
                Map.of("input_ids", tIds, "attention_mask", tMask, "token_type_ids", tTypes))) {

            float[][][] hidden = (float[][][]) res.get(0).getValue(); // [batch][seq][384]
            List<float[]> out = new ArrayList<>(enc.length);
            for (int b = 0; b < enc.length; b++) out.add(Pooling.meanAndNormalize(hidden[b], mask[b]));
            return out;
        } catch (OrtException e) {
            throw new IllegalStateException("ONNX inference failed", e);
        }
    }

    @Override public void close() throws OrtException { session.close(); }
}

```

Option B: Hugging Face Inference API. Endpoint (from a Hugging Face forum thread on the 2026 router migration): `POST https://router.huggingface.co/hf-inference/models/{model}/pipeline/feature-extraction`, header `Authorization: Bearer <HF_TOKEN>`, body `{"inputs": ["text1", "text2"]}`. The OpenAI-style `/v1/embeddings` route does **not** serve embeddings there. A model that no inference provider deploys returns 404, which a JSON client reports as a

confusing parse error. The response shape can vary by model, so the parser accepts both a [n][384] and a per-token [n][tokens][384] answer. I could not call the API from this sandbox, so test it with one curl first.

HfApiEmbeddingClient.java (RestClient; the same calls work with RestTemplate or WebClient)

```
// HfApiEmbeddingClient.java -- Option B: Hugging Face Inference API over HTTPS (RestClient)
import com.fasterxml.jackson.databind.JsonNode;
import java.time.Duration;
import java.util.*;
import org.springframework.http.*;
import org.springframework.http.client.SimpleClientHttpRequestFactory;
import org.springframework.web.client.RestClient;

@Service
@ConditionalOnProperty(name = "rag.embedder", havingValue = "hf-api")
public class HfApiEmbeddingClient implements EmbeddingClient {

    private static final int BATCH = 16;
    private final RestClient http;
    public HfApiEmbeddingClient(RestClient.Builder builder,
                               @Value("${hf.token}") String token,
                               @Value("${hf.model:sentence-transformers/all-MiniLM-L6-v2}") String model) {
        var rf = new SimpleClientHttpRequestFactory();
        rf.setConnectTimeout(Duration.ofSeconds(3)); // rule 3 of the ShopNest story: every call has a timeout
        rf.setReadTimeout(Duration.ofSeconds(60));
        this.http = builder.requestFactory(rf)
            .baseUrl("https://router.huggingface.co/hf-inference/models/" + model + "/pipeline/feature-extraction")
            .defaultHeader(HttpHeaders.AUTHORIZATION, "Bearer " + token)
            .build();
    }

    @Override
    public List<float[]> embedAll(List<String> texts) {
        List<float[]> out = new ArrayList<>(texts.size());
        for (List<String> batch : Batches.partition(texts, BATCH)) {
            JsonNode root = http.post()
                .contentType(MediaType.APPLICATION_JSON)
                .body(Map.of("inputs", batch)) // {"inputs": ["text 1", "text 2", ...]}
                .retrieve()
                .body(JsonNode.class);
            out.addAll(parse(root, batch.size()));
        }
        return out;
    }

    /** Expected: [[384 floats], ...] one row per input. If a model returns per-token vectors ([[384]..]), pool them. */
    private List<float[]> parse(JsonNode root, int expected) {
        if (root == null || !root.isArray() || root.size() != expected)
            throw new IllegalStateException("Unexpected HF response: " + root);
        List<float[]> out = new ArrayList<>(expected);
        for (JsonNode item : root) {
            if (item.get(0).isArray()) { // 3-D: tokens x 384
                float[][] tokens = new float[item.size()][];
                long[] ones = new long[item.size()];
                Arrays.fill(ones, 1);
                for (int t = 0; t < tokens.length; t++) tokens[t] = toFloats(item.get(t));
                out.add(Pooling.meanAndNormalize(tokens, ones));
            } else {
                out.add(Pooling.normalize(toFloats(item))); // 2-D: already one vector per input
            }
        }
        for (float[] v : out)
            if (v.length != DIMENSIONS) throw new IllegalStateException("Expected 384 dims, got " + v.length);
        return out;
    }

    private static float[] toFloats(JsonNode arr) {
        float[] v = new float[arr.size()];
        for (int i = 0; i < v.length; i++) v[i] = (float) arr.get(i).asDouble();
        return v;
    }
}
```

Option C: Python FastAPI service, Spring Boot as the client. Python does only what it is best at (running sentence-transformers); Java owns batching, retries and storage.

Python service (about 20 lines)

```
# embed_service.py      run:  pip install fastapi uvicorn sentence-transformers
#                        uvicorn embed_service:app --port 8001
from fastapi import FastAPI
from pydantic import BaseModel
from sentence_transformers import SentenceTransformer

app = FastAPI()
model = SentenceTransformer("sentence-transformers/all-MiniLM-L6-v2") # downloads ~90 MB the first time
model.max_seq_length = 256 # the default, written out loud

class EmbedRequest(BaseModel):
    texts: list[str]
@app.post("/embed")
def embed(req: EmbedRequest):
    vecs = model.encode(req.texts, batch_size=32, normalize_embeddings=True) # numpy array [n, 384]
    return {"model": "all-MiniLM-L6-v2", "dim": int(vecs.shape[1]), "count": len(req.texts),
            "embeddings": vecs.tolist()}
```

Complete request / response JSON

```
POST http://localhost:8001/embed      Content-Type: application/json

REQUEST  (16 chunks per call; shown with 2)
{ "texts": [ "ShopNest began in February 2022 in a rented two-room flat in Kondapur, Hyderabad. ...",
             "The team wrote down three rules in a file called PRINCIPLES.md ..." ] }

RESPONSE (one 384-float row per input, SAME ORDER as the request)
{ "model": "all-MiniLM-L6-v2", "dim": 384, "count": 2,
  "embeddings": [ [ 0.0412, -0.0187, 0.0953, ... 381 more numbers ... ],
                  [ -0.0066, 0.0331, 0.0120, ... 381 more numbers ... ] ] }
```

IngestionService has 50 chunks -> FastApiEmbeddingClient splits them into 4 requests



responses are appended in order: embeddings.get(i) belongs to chunks.get(i) - never sort, never parallelise blindly

FastApiEmbeddingClient.java (batching 50 chunks, retry with backoff; not compiled here)

```
// FastApiEmbeddingClient.java -- Option C: Spring Boot calls the Python service above
import java.time.Duration;
import java.util.*;
import org.springframework.http.MediaType;
import org.springframework.http.client.SimpleClientHttpRequestFactory;
import org.springframework.web.client.*;

@Service
@ConditionalOnProperty(name = "rag.embedder", havingValue = "fastapi")
public class FastApiEmbeddingClient implements EmbeddingClient {

    record EmbedRequest(List<String> texts) {}
    record EmbedResponse(String model, int dim, int count, float[][] embeddings) {} // Jackson maps float[][] directly
    private static final int BATCH = 16;
    private final RestClient http;

    public FastApiEmbeddingClient(RestClient.Builder builder,
                                @Value("${rag.fastapi.url:http://localhost:8001}") String url) {
        var rf = new SimpleClientHttpRequestFactory();
        rf.setConnectTimeout(Duration.ofSeconds(2));
        rf.setReadTimeout(Duration.ofSeconds(30));
        this.http = builder.requestFactory(rf).baseUrl(url).build();
    }

    /** 50 chunks -> 4 HTTP calls of 16 + 16 + 16 + 2, results concatenated in order. */
    @Override
    public List<float[]> embedAll(List<String> texts) {
        List<float[]> out = new ArrayList<>(texts.size());
        for (List<String> batch : Batches.partition(texts, BATCH)) {
            EmbedResponse r = callWithRetry(new EmbedRequest(batch));
            if (r == null || r.dim() != DIMENSIONS || r.embeddings().length != batch.size())
                throw new IllegalStateException("Bad embed response: expected " + batch.size() + " x 384");
            for (float[] v : r.embeddings()) out.add(v);
        }
        return out;
    }

    private EmbedResponse callWithRetry(EmbedRequest req) {
        RuntimeException last = null;
        for (int attempt = 0; attempt < 3; attempt++) {
            try {
```

```

        return http.post().uri("/embed").contentType(MediaType.APPLICATION_JSON)
            .body(req).retrieve().body(EmbedResponse.class);
    } catch (ResourceAccessException | HttpServerErrorException e) { // timeout / connection / 5xx
        last = e;
        try { Thread.sleep(300L << attempt); }
        catch (InterruptedException ie) { Thread.currentThread().interrupt(); }
    }
}
throw new IllegalStateException("Embedding service unavailable", last);
}
}

```

Option D (your lab today): wrap the bean you already have so it satisfies the same interface, then run the Step 6-7 pipeline on it first. It is the quickest route to a working end-to-end flow, and later the oracle you compare Option A against.

SpringAiEmbeddingClient.java

```

// SpringAiEmbeddingClient.java -- Option D: reuse the bean your lab ALREADY has (TransformersEmbeddingModel)
import org.springframework.ai.embedding.EmbeddingModel;

@Service
@ConditionalOnProperty(name = "rag.embedder", havingValue = "spring-ai", matchIfMissing = true)
public class SpringAiEmbeddingClient implements EmbeddingClient {

    private final EmbeddingModel model; // injected: TransformersEmbeddingModel (all-MiniLM-L6-v2)

    public SpringAiEmbeddingClient(EmbeddingModel model) { this.model = model; }

    @Override
    public List<float[]> embedAll(List<String> texts) {
        return model.embed(texts); // verify in Spring AI 2.0 (List<float[]> in 1.x)
    }
}

```

Why this matters

The interface keeps the model swappable: you can start on D, prove the pipeline, then replace it with A or C and re-run the same 10 questions. Batching matters because a model call has fixed overhead; 16 texts per call is a sensible start (tune by measuring).

What goes wrong

- **Embedding documents with one model and questions with another** (or the same model through a path with different pooling/normalisation): the dimension may even match, and results are plausible-looking nonsense. Always one embedder bean, used for both.
- **No timeouts** on the HTTP clients: a slow embedding service becomes your outage (the same lesson as the story's Chapter 5). All clients above set connect and read timeouts.
- **All 50 chunks in one request** risks payload and timeout limits; **one request per chunk** pays 50 round trips. **Losing order** (parallel calls, sorting) attaches vectors to the wrong text.
- **Holding a DB transaction open** while waiting on the network starves HikariCP: embed first, then open a short transaction (see IngestionService in Step 7).

STEP 6 PGVECTOR STORAGE: table, JPA entity, top-3 cosine query

Operator	Meaning	Use when	Index operator class
<=>	cosine distance (0 same, 1 unrelated, 2 opposite)	text embeddings (our case). similarity = 1 - distance	vector_cosine_ops
<->	Euclidean (L2) distance	when vector length carries meaning	vector_l2_ops
<#>	negative inner product	normalised vectors, fastest; remember the minus sign	vector_ip_ops

schema.sql

```
-- schema.sql (run once; your docker-compose pgvector image already ships the extension)
CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE IF NOT EXISTS shopnest_chunks (
    id          BIGSERIAL      PRIMARY KEY,
    doc_id      TEXT          NOT NULL,
    chunk_index INT           NOT NULL,
    content     TEXT          NOT NULL,
    token_count INT           NOT NULL,
    embedding   vector(384)    NOT NULL,          -- 384 = hidden size of all-MiniLM-L6-v2. Wrong size = INSERT fails.
    created_at  TIMESTAMPTZ   NOT NULL DEFAULT now(),
    UNIQUE (doc_id, chunk_index)
);

-- Optional at 71 rows (Postgres will seq-scan and give EXACT results). Worth it from ~50k+ rows.
-- Defaults: m = 16, ef_construction = 64. Query-time knob: SET hnsw.ef_search = 40 (default).
CREATE INDEX IF NOT EXISTS idx_shopnest_chunks_embedding
    ON shopnest_chunks USING hnsw (embedding vector_cosine_ops);

-- Top 3 for a query vector (psql, using chunk 29's own vector as the "question"):
SELECT chunk_index,
       1 - (embedding <=> q.v) AS similarity          -- cosine distance -> similarity
FROM shopnest_chunks, (SELECT embedding AS v FROM shopnest_chunks WHERE chunk_index = 29) q
ORDER BY embedding <=> q.v                          -- ORDER BY the SAME expression the index was built for
LIMIT 3;
```

JPA mapping. Hibernate 6.4+ (Spring Boot 4.x ships Hibernate 7) can map a `float[]` to a pgvector column with `@JdbcTypeCode(SqlTypes.VECTOR)` and `@Array(length = 384)`; it needs the `org.hibernate.orm:hibernate-vector` dependency at the same version as `hibernate-core` (if Maven cannot resolve the version, set it explicitly). The similarity search itself is a native query, because `<=>` is not JPQL. The query vector travels as a text literal `"[0.1,0.2,...]"` and is cast on the database side.

ChunkEntity.java (not compiled here)

```
// ChunkEntity.java (pom: org.hibernate.orm:hibernate-vector, same version as your hibernate-core)
import jakarta.persistence.*;
import org.hibernate.annotations.Array;
import org.hibernate.annotations.JdbcTypeCode;
import org.hibernate.type.SqlTypes;

@Entity
@Table(name = "shopnest_chunks")
public class ChunkEntity {

    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "doc_id", nullable = false) private String docId;
    @Column(name = "chunk_index", nullable = false) private int chunkIndex;
    @Column(nullable = false, columnDefinition = "text") private String content;
    @Column(name = "token_count", nullable = false) private int tokenCount;

    @JdbcTypeCode(SqlTypes.VECTOR) // Hibernate 6.4+/7: maps float[] <-> pgvector "vector"
    @Array(length = 384) // dimension
    @Column(name = "embedding", nullable = false)
    private float[] embedding;

    protected ChunkEntity() {} // JPA needs a no-arg constructor
    public ChunkEntity(String docId, int chunkIndex, String content, int tokenCount, float[] embedding) {
        this.docId = docId; this.chunkIndex = chunkIndex; this.content = content;
        this.tokenCount = tokenCount; this.embedding = embedding;
    }
    // getters ...
}
```


ChunkRepository.java with the top-3 native query (not compiled here)

```
// ChunkRepository.java
import java.util.List;
import org.springframework.data.jpa.repository.*;
import org.springframework.data.repository.query.Param;

public interface ChunkRepository extends JpaRepository<ChunkEntity, Long> {

    /** Interface projection: Spring Data fills it from the SELECT aliases below. */
    interface ChunkHit {
        Long getId();
        String getDocId();
        Integer getChunkIndex();
        String getContent();
        Double getScore(); // cosine similarity, 1.0 = identical direction
    }

    long deleteByDocId(String docId);

    // Aliases are QUOTED on purpose: Postgres lower-cases unquoted aliases and the camelCase getters would stay null.
    @Query(value = """
        SELECT c.id AS "id", c.doc_id AS "docId", c.chunk_index AS "chunkIndex", c.content AS "content",
               1 - (c.embedding <=> CAST(:q AS vector)) AS "score"
        FROM shopnest_chunks c
        ORDER BY c.embedding <=> CAST(:q AS vector)
        LIMIT :k
        """, nativeQuery = true)
    List<ChunkHit> findTopK(@Param("q") String queryVector, @Param("k") int k);
}
```

Plan B: no JPA, same table, same query

```
// PLAN B if hibernate-vector gives you trouble: plain JdbcTemplate, no JPA magic, same table
jdbc.update("""
INSERT INTO shopnest_chunks (doc_id, chunk_index, content, token_count, embedding)
VALUES (?, ?, ?, ?, CAST(? AS vector))
ON CONFLICT (doc_id, chunk_index) DO UPDATE
SET content = EXCLUDED.content, token_count = EXCLUDED.token_count, embedding = EXCLUDED.embedding
""", docId, i, text, tokens, VectorLiteral.of(vec));

List<Map<String, Object>> top3 = jdbc.queryForList("""
SELECT chunk_index, content, 1 - (embedding <=> CAST(? AS vector)) AS score
FROM shopnest_chunks ORDER BY embedding <=> CAST(? AS vector) LIMIT 3
""", VectorLiteral.of(q), VectorLiteral.of(q));
```

Cross-check with your existing code: run the same question through `PgVectorStore.similaritySearch(...)` (its API changed between Spring AI versions, check 2.0) and through `findTopK`. With the same embedder both should return the same chunks. If they differ, one pipeline has a bug, and you just found it.

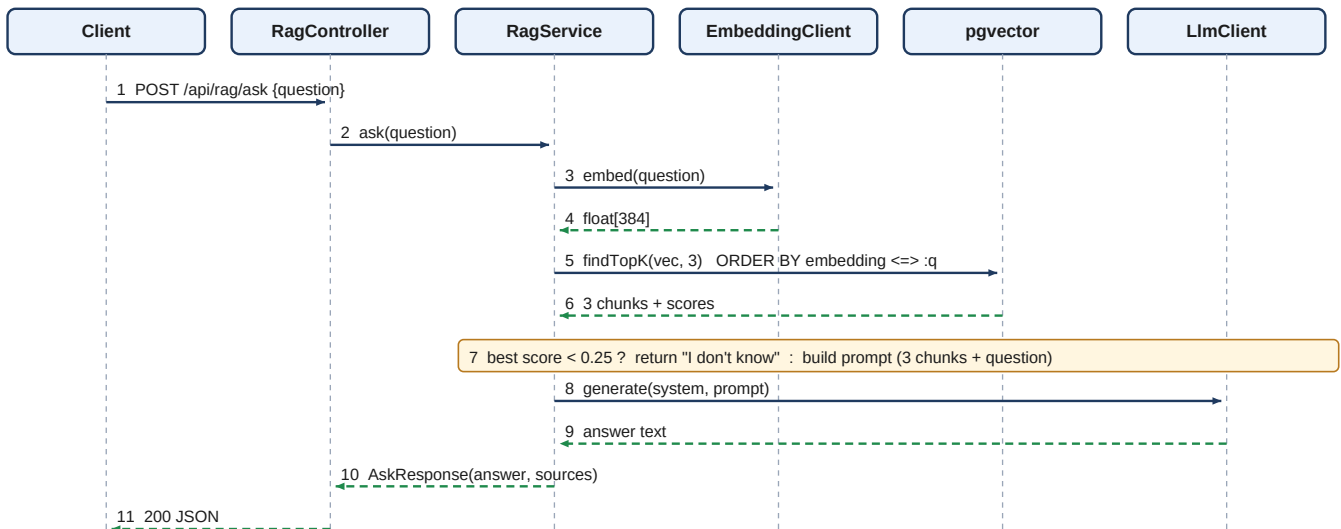
Why this matters

The column type `vector(384)` is a contract: Postgres rejects any insert that is not exactly 384 floats, which catches a wrong-model mistake early. The distance expression in `ORDER BY` must match the index's operator class, or Postgres falls back to scanning every row. At 71 rows that is instant and exact, so the HNSW index is optional for this lab; it starts to matter from tens of thousands of rows.

What goes wrong

- **Wrong dimension** (a 768-d model into `vector(384)`): the insert fails with a dimension error. Good: fail loudly.
- **Unquoted camelCase aliases** in a native query on Postgres: the projection getters come back null (aliases are lower-cased). The query above quotes them.
- **Index built with cosine ops but queried with `<=>`** (or the reverse): correct results, no index use, slow at scale.
- **Reading distance as similarity:** distance 0.1 is very close, similarity 0.1 is far. Always report `1 - distance` and name it clearly.
- **Approximate index** (HNSW) can miss a true neighbour: fine for retrieval, so always evaluate on your own questions (Step 8).

STEP 7 FULL RAG FLOW: Controller -> embed -> pgvector top 3 -> LLM -> answer



Ingestion (one-off, per document) and the **query path** share the same embedder, chunker settings and table. Two design choices worth copying: (1) a **score threshold** so an irrelevant question never reaches the LLM (the CFO question), and (2) embedding happens **outside** any DB transaction.

RagController.java (not compiled here)

```

// RagController.java + DTOs
public record AskRequest(@NotBlank String question) {}
public record Source(int chunkIndex, double score, String preview) {}
public record AskResponse(String answer, List<Source> sources) {}

@RestController
@RequestMapping("/api/rag")
public class RagController {

    private final IngestionService ingestion;
    private final RagService rag;

    public RagController(IngestionService ingestion, RagService rag) { this.ingestion = ingestion; this.rag = rag; }

    @PostMapping(value = "/ingest", consumes = MediaType.TEXT_PLAIN_VALUE)
    public Map<String, Object> ingest(@RequestParam String docId, @RequestBody String text) {
        return Map.of("docId", docId, "chunks", ingestion.ingest(docId, text));
    }

    @PostMapping("/ask")
    public AskResponse ask(@Valid @RequestBody AskRequest req) {
        return rag.ask(req.question());
    }
}
    
```

IngestionService.java + ChunkStore.java (not compiled here)

```

// IngestionService.java + ChunkStore.java (ingest = chunk -> embed -> store)
@Service
public class IngestionService {

    private final SentenceChunker chunker = new SentenceChunker(1000, 150);
    private final EmbeddingClient embedder;
    private final WordPiece tokenizer;
    private final ChunkStore store;

    public IngestionService(EmbeddingClient embedder, WordPiece tokenizer, ChunkStore store) {
        this.embedder = embedder; this.tokenizer = tokenizer; this.store = store;
    }

    public int ingest(String docId, String text) {
        List<String> chunks = chunker.chunk(text);
        List<float[]> vectors = embedder.embedAll(chunks);
        if (vectors.size() != chunks.size()) throw new IllegalStateException("vector count != chunk count");
        List<ChunkEntity> rows = new ArrayList<>();
        for (int i = 0; i < chunks.size(); i++) {
            int tokens = tokenizer.countTokens(chunks.get(i));
            if (tokens > 256) log.warn("chunk {} has {} tokens: the model will truncate it", i, tokens);
            rows.add(new ChunkEntity(docId, i, chunks.get(i), tokens, vectors.get(i)));
        }
        store.replace(docId, rows);
        return rows.size();
    }
}
    
```

```

@Service
class ChunkStore {
    private final ChunkRepository repo;
    ChunkStore(ChunkRepository repo) { this.repo = repo; }

    @Transactional
    void replace(String docId, List<ChunkEntity> rows) { repo.deleteByDocId(docId); repo.saveAll(rows); }
}

```

RagService.java

```

// RagService.java -- the whole query path in one readable method
@Service
public class RagService {

    private static final int TOP_K = 3;
    private static final double MIN_SCORE = 0.25; // a starting guess: tune it on your eval questions (Step 8)
    private static final String NO_ANSWER = "I don't know based on the provided documents.";

    private final EmbeddingClient embedder;
    private final ChunkRepository repo;
    private final LlmClient llm;

    public RagService(EmbeddingClient embedder, ChunkRepository repo, LlmClient llm) {
        this.embedder = embedder; this.repo = repo; this.llm = llm;
    }

    public AskResponse ask(String question) {
        float[] q = embedder.embed(question); // 1. question -> 384 floats
        List<ChunkRepository.ChunkHit> hits = repo.findTopK(VectorLiteral.of(q), TOP_K); // 2. nearest 3 chunks
        log.info("q='{ }' hits={ }", question, hits.stream().map(h -> h.getChunkIndex() + ":" + h.getScore()).toList());

        if (hits.isEmpty() || hits.get(0).getScore() < MIN_SCORE) // 3. nothing relevant? skip LLM
            return new AskResponse(NO_ANSWER, List.of());

        String answer = llm.generate(PromptBuilder.SYSTEM, PromptBuilder.build(question, hits)); // 4. generate
        List<Source> sources = hits.stream()
            .map(h -> new Source(h.getChunkIndex(), h.getScore(), preview(h.getContent()))).toList();
        return new AskResponse(answer, sources); // 5. answer + where it came from
    }

    private static String preview(String s) { return s.length() <= 120 ? s : s.substring(0, 120) + "..."; }
}

```

PromptBuilder.java

```

// PromptBuilder.java -- the prompt is just a String template
public final class PromptBuilder {
    private PromptBuilder() {}

    public static final String SYSTEM = """
        You are the ShopNest knowledge assistant. Answer ONLY from the CONTEXT blocks below.
        If the answer is not in the context, reply exactly: I don't know based on the provided documents.
        Cite the chunk numbers you used, like [chunk 12]. Be concise."";

    public static String build(String question, List<ChunkRepository.ChunkHit> hits) {
        StringBuilder sb = new StringBuilder("CONTEXT:\n");
        for (var h : hits) {
            sb.append("[chunk ").append(h.getChunkIndex()).append("\n").append(h.getContent()).append("\n\n");
        }
        return sb.append("QUESTION: ").append(question).append("\nANSWER:").toString();
    }
}

```

What the LLM receives for the double-charge question (chunk 15 text is real; the other two depend on your model's scores)

```

SYSTEM: You are the ShopNest knowledge assistant. Answer ONLY from the CONTEXT blocks below. ...

CONTEXT:
[chunk 15]
The checkout call to create a Razorpay order used a RestTemplate with a 30 second read timeout, and the code
retried up to three times on any exception. Gateways occasionally respond slowly under load, so a call that
timed out on the ShopNest side might still have succeeded on the Razorpay side. On the evening of 14 October
2022 ...

[chunk N]
...second most similar chunk...

[chunk M]
...third most similar chunk...
QUESTION: How many customers were charged twice, and why?
ANSWER:

```

LlmClient.java + GeminiClient.java (REST shape of generateContent; model name comes from config)

```
// LlmClient.java + GeminiClient.java (swap the implementation, keep RagService untouched)
public interface LlmClient { String generate(String system, String user); }

@Service
public class GeminiClient implements LlmClient {

    private final RestClient http;
    private final String model;

    public GeminiClient(RestClient.Builder b, @Value("${gemini.api-key}") String key,
        @Value("${gemini.model}") String model) {
        this.model = model;
        var rf = new SimpleClientHttpRequestFactory();
        rf.setConnectTimeout(Duration.ofSeconds(3));
        rf.setReadTimeout(Duration.ofSeconds(60));
        this.http = b.requestFactory(rf)
            .baseUrl("https://generativelanguage.googleapis.com/v1beta")
            .defaultHeader("x-goog-api-key", key) // key from env var GEMINI_API_KEY, never from git
            .build();
    }

    @Override
    public String generate(String system, String user) {
        Map<String, Object> body = Map.of(
            "systemInstruction", Map.of("parts", List.of(Map.of("text", system))),
            "contents", List.of(Map.of("role", "user", "parts", List.of(Map.of("text", user)))),
            "generationConfig", Map.of("temperature", 0.1)); // low = stick to the context
        JsonNode r = http.post().uri("/models/{m}:generateContent", model)
            .contentType(MediaType.APPLICATION_JSON).body(body).retrieve().body(JsonNode.class);
        JsonNode text = r.at("/candidates/0/content/parts/0/text");
        if (text.isMissingNode()) throw new IllegalStateException("Gemini returned no text (safety block?): " + r);
        return text.asText();
    }
}
```

If you use OpenAI instead	Gemini (above)	OpenAI
Endpoint	POST .../v1beta/models/{model}:generateContent	POST https://api.openai.com/v1/chat/completions
Auth header	x-goog-api-key: KEY	Authorization: Bearer KEY
System prompt	systemInstruction.parts[].text	messages[0] = {role: system, content}
User prompt	contents[].parts[].text	messages[1] = {role: user, content}
Answer at	/candidates/0/content/parts/0/text	/choices/0/message/content

application.yml additions + RagConfig

```
# application.yml (additions)
rag:
  embedder: onnx # onnx | hf-api | fastapi | spring-ai
  vocab: ${RAG_VOCAB:./models/vocab.txt}
  onnx:
    model: ${RAG_ONNX_MODEL:./models/model.onnx}
  hf:
    token: ${HF_TOKEN:}
  gemini:
    api-key: ${GEMINI_API_KEY:}
    model: ${GEMINI_MODEL:} # pick a current Flash-class model from the Gemini docs
  spring:
    jpa:
      hibernate:
        ddl-auto: validate # schema.sql owns the table, Hibernate only checks it

// RagConfig.java
@Configuration
class RagConfig {
    @Bean WordPiece wordPiece(@Value("${rag.vocab}") Path vocab) throws IOException { return new WordPiece(vocab); }
}
```

Try it (two curl calls):

Smoke test

```
curl -X POST "localhost:8080/api/rag/ingest?docId=shopnest" -H "Content-Type: text/plain" --data-binary @shopnest_story.txt
-> {"docId":"shopnest","chunks":71}

curl -X POST localhost:8080/api/rag/ask -H "Content-Type: application/json" \
-d '{"question":"How many customers were charged twice, and why?"}'
-> expect: 37 customers, because payment creation was not idempotent, sources include chunk 15

curl ... -d '{"question":"Who is the CFO of ShopNest?"}'
-> expect: "I don't know based on the provided documents." (score below threshold or the prompt rule)
```

Why this matters

Every stage can fail independently, so build and test them separately: 1) is the right chunk retrieved? 2) does the LLM answer correctly from it? The response returns **sources** and the service **logs chunk ids and scores** so you can answer question 1 from the logs alone.

What goes wrong

- **No 'answer only from context' instruction:** the LLM fills gaps from its training data and confidently invents a CFO. Pair the instruction with a score threshold.
- **top_k = 1:** one unlucky ranking loses the answer. **top_k = 10:** ~2,500 tokens of noise, higher cost, and the model gets distracted. Start at 3-5 and measure.
- **MIN_SCORE too high** refuses valid questions; too low lets irrelevant chunks through. 0.25 is a placeholder: set it from your 10 questions (Step 8).
- **High temperature** makes answers drift from the context; use ~0.1 for question answering.
- **API keys in git** or in application.yml: read them from environment variables (the config above does). Calling the LLM inside a DB transaction ties up a connection for seconds.

STEP 8 LEARNING FOCUS: failure atlas, eval harness, build order

Every mistake in this guide, with the number measured on the ShopNest document:

Mistake	Symptom you would see	Measured on the 50,066-char doc
No splitting	most questions return nothing relevant	model sees 2.5% of the text (256 of 10,185 tokens)
Chunks of 5000 chars	results come back, but are blind to most of each chunk	10 chunks, all over 256 tokens, 25% embedded
Chunks of 3000 chars	same, a bit milder	17 chunks, 43% embedded
Chunks of 200 chars	fragments without context; 'it' has no referent	250 chunks, avg 44 tokens
Fixed windows, no overlap	facts cut in half, found nowhere	46 of 387 sentences (12%) never whole; 48 chunks start mid-sentence
Fixed windows + char overlap	no more lost sentences, but chunks start mid-word	58 of 59 chunks start mid-sentence
Naive ' ' sentence split	'Dr.' and '6,400.' become sentences	392 'sentences' vs the real 387
Truncation ignored	tail of a chunk silently missing	542-token text: 286 tokens (52%) dropped, mid-word
Padding included in the mean	vector depends on batch composition	toy case: 2.42 vs 0.55 for the first component
Different model for query and docs	plausible-looking nonsense	no error, so only your eval set catches it
No score threshold, no 'only from context'	confident answers to unanswerable questions	the CFO question

Two-stage debugging for any wrong answer: look at the logged chunk ids. **Right chunk absent** → chunking, tokenization, embedder or index problem (Steps 3-6). **Right chunk present, answer still wrong** → prompt or LLM problem (Step 7). Never tune the prompt when the retrieval is broken.

Three tests that catch most embedding bugs

```
// The three tests that catch 90% of embedding bugs
// 1) Golden vector: compare Java against Python for a fixed sentence (first 5 numbers, tolerance 1e-3)
// python -c "from sentence_transformers import SentenceTransformer as S;
// print(S('all-MiniLM-L6-v2').encode(['Diwali sale outage'], normalize_embeddings=True)[0][:5])"
@Test void goldenVector() { float[] v = embedder.embed("Diwali sale outage"); /* assert v[0..4] ~ python values */ }

// 2) Shape + length: 384 floats, unit length
@Test void shapeAndNorm() {
    float[] v = embedder.embed("hello");
    assertEquals(384, v.length);
    assertEquals(1.0, Math.sqrt(IntStream.range(0, 384).mapToDouble(i -> v[i] * v[i]).sum()), 1e-4);
}

// 3) Self-retrieval: embed chunk 29's own text, search -> chunk 29 must be #1 with score ~1.0
@Test void selfRetrieval() {
    var top = repo.findTopK(VectorLiteral.of(embedder.embed(chunk29Text)), 3);
    assertEquals(29, top.get(0).getChunkIndex());
    assertTrue(top.get(0).getScore() > 0.99);
}
```

RagEvalIT.java: hit@3 over your 10 questions (record the first number you get as the baseline in 03-rag/README.md)

```
// RagEvalIT.java -- your own "unit tests" for retrieval quality (needs DB + embedder, so run it as an integration test)
@SpringBootTest
class RagEvalIT {

    record Case(String question, String mustContain) {}

    static final List<Case> CASES = List.of(
        new Case("What was the first order on ShopNest and how much was it?", "Guntur chillies"),
        new Case("How many customers were charged twice, and why?", "Thirty seven customers"),
        new Case("How did the team fix the N+1 query problem?", "JOIN FETCH"),
        new Case("How long did the Diwali outage last?", "The outage lasted 47 minutes"),
        new Case("How many partitions does the orders topic have?", "12 partitions"),
        new Case("What availability target did the team set for checkout?", "99.9 percent"),
        new Case("How much did the AWS bill fall by, and from what?", "Rs. 9.6 lakh to Rs. 5.8 lakh"),
```

```

        new Case("Which embedding model and how many dimensions were used?", "all-MiniLM-L6-v2"),
        new Case("How much did ShopNest raise in its Series A?", "Rs. 42 crore"),
        new Case("What was the real return policy?", "seven days"));

@Autowired EmbeddingClient embedder;
@Autowired ChunkRepository repo;

@Test
void hitRateAtThree() {
    int hits = 0;
    for (Case c : CASES) {
        var top = repo.findTopK(VectorLiteral.of(embedder.embed(c.question()), 3);
        boolean hit = top.stream().anyMatch(h -> h.getContent().contains(c.mustContain()));
        System.out.printf("%-4s %-62s top3=%s\n", hit ? "HIT" : "MISS", c.question(),
            top.stream().map(h -> h.getChunkIndex() + ":" + String.format("%.2f", h.getScore())).toList());
        if (hit) hits++;
    }
    double rate = hits / (double) CASES.size();
    System.out.println("hit@3 = " + rate);
    assertTrue(rate >= 0.8, "retrieval regressed: hit@3 = " + rate);
}
}

```

Plain-Java self-test output (what I ran against the real document):

SelfTest

```

PASS 71 chunks for the 50,066-char doc
PASS no chunk longer than 1000 chars
PASS no chunk is blank
PASS every sentence is whole inside at least one chunk
PASS every consecutive pair overlaps by >= 150 chars
PASS no chunk exceeds 256 tokens
PASS 'Dr. Meera Rao' and 'Rs. 18 lakh' are not split
PASS HikariCP -> [hi, ##kari, ##cp]
PASS encode truncates to 256 and keeps [SEP]
PASS pooling ignores [PAD] rows
PASS pooled vector has unit length
PASS partition(50, 16) -> [16,16,16,2]
PASS vector literal format
PASS vector literal rejects NaN
ALL PASSED

```

Suggested build order for 03-rag (each step ends green before the next starts):

#	Milestone	Done when
3.1	Copy SentenceChunker, WordPiece, Pooling, Batches, VectorLiteral into 03-rag. Add the JUnit tests.	chunker tests pass on shopnest_story.txt: 71 chunks, none over 256 tokens
3.2	Embedder behind EmbeddingClient: start with Option D (your Spring AI bean), then A or C.	shape + norm + golden-vector tests pass (Python sentence-transformers is the oracle)
3.3	schema.sql, ChunkEntity, ChunkRepository, IngestionService.	ingest returns 71; self-retrieval test: chunk 29 is its own top-1 with score above 0.99
3.4	Retrieval-only endpoint (no LLM). Compare with PgVectorStore results.	answer-key table: the expected chunk ids appear in the top 3
3.5	PromptBuilder, GeminiClient, RagService, controller.	curl smoke tests: 37 customers answered, CFO question refused
3.6	RagEvalIT with the 10 questions; pick TOP_K and MIN_SCORE from the results; write the baseline hit@3 into the README.	a number you can beat when you change chunk size, model or prompt

Sources. Hugging Face forum thread on the 2026 Serverless Inference router (endpoint and 404 behaviour): discuss.huggingface.co/t/404-not-found-unexpected-token-n-on-serverless-inference-router-2026-migration/173639. ai-engineering-lab README (stack and milestone status): github.com/ARGOD2213/ai-engineering-lab. Model facts (384 dimensions, 256 word-piece default, uncased BERT vocabulary) are from the all-MiniLM-L6-v2 model card as I know it; confirm on its Hugging Face page. Everything quantitative in this PDF was measured by running the code listed here.